Problem: Bisection Method

Theory:

The Bisection Method is a numerical technique used to find roots of a continuous function f(x). It is based on the Intermediate Value Theorem, which states that if a continuous function changes sign over an interval [xl, xu], then it must have at least one root in that interval.

Steps of the method:

1. Choose an interval [xl, xu] such that f(xl) * f(xu) < 0.
2. Compute the midpoint: xr = (xl + xu) / 2.
3. Evaluate f(xr).
    - If f(xr) = 0, xr is the root.
    - If f(xl) * f(xr) < 0, the root lies in [xl, xr]. Set xu = xr.
    - If f(xl) * f(xr) > 0, the root lies in [xr, xu]. Set xl = xr.
4. Repeat steps 2–3 until the approximate error is less than the desired tolerance (epsilon) or the maximum number of iterations is reached.

The method is simple and reliable, but convergence is linear and may require many iterations for high precision.

Code:

Bisection method.py

Bisection method.py > Bisection

```python
def f(x):
    return 0.5*x*x*x - x*x

def Bisection(f, xl, xu, max_itr = 300, eps = 0.05):
    if f(xl)*f(xu)>0:
        print("Wrong guess!")
        return None

    if f(xl)*f(xu) == 0:
        if f(xl) == 0:
            return xl
        else:
            return xu

    itr = 1
    xr_old = (xl + xu)/2
    while True:
        if f(xl) * f(xr_old) > 0:
            xl = xr_old
        elif f(xl)* f(xr_old) < 0:
            xu = xr_old
```

```
            xu = xr_old
        else:
            return xr_old
        xr_new = (xl+xu)/2
        ae = abs(xr_new - xr_old)
        xr_old = xr_new

        itr = itr+1
        if ae <= eps or itr>max_itr:
            break
    return xr_old

xl, xu = 1,3
xr = Bisection(f, xl, xu)

if xr is not None:
    print(f"The root of this function: {xr:.3f}")
    print(f"The value of the function is f(xr): {f(xr):.3f}")
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\@CODES\Academic_Labs\CSE 2206 - Numerical Methods Lab\Lab 3> & C:/Us
ical Methods Lab/Lab 3/Bisection method.py"
The root of this function: 2.000
The value of the function is f(xr): 0.000
PS E:\@CODES\Academic_Labs\CSE 2206 - Numerical Methods Lab\Lab 3>
```

Conclusion:

- The Bisection Method successfully found the root of the function f(x) = 0.5*x^3 - x^2 within the interval [1, 3].
- The approximate root is xr ≈ 2.000, and f(xr) ≈ 0, confirming accuracy.
- The method is robust since it guarantees convergence if the initial interval brackets a root, though it may require multiple iterations for better precision.

Problem: False Position Method

Theory:

The False Position Method (Regula Falsi) is a numerical technique to find roots of a continuous function f(x). Like the Bisection Method, it requires an interval [xl, xu] such that f(xl) * f(xu) < 0, which guarantees at least one root in that interval.

Instead of using the midpoint, the False Position Method estimates the root using a straight line connecting the points (xl, f(xl)) and (xu, f(xu)). The formula for the next approximation is:

xr = xu - (f(xu) * (xl - xu)) / (f(xl) - f(xu))

Steps of the method:

1. Choose initial guesses xl and xu such that f(xl) * f(xu) < 0.
2. Compute xr using the formula above.
3. Evaluate f(xr).
    - If f(xr) = 0, xr is the root.
    - If f(xl) * f(xr) < 0, set xu = xr.
    - If f(xl) * f(xr) > 0, set xl = xr.
4. Repeat steps 2–3 until the approximate error is less than a desired tolerance (epsilon) or maximum iterations are reached.

This method usually converges faster than the Bisection Method because it uses a more accurate approximation of the root.

Code:

False position method.py > FalsePosition

```python
def f(x):
    return 0.5*x*x*x - x*x

def FalsePosition(f, xl, xu, max_itr=300, eps=0.05):
    if f(xl) * f(xu) > 0:
        print("Wrong guess!")
        return None

    if f(xl) * f(xu) == 0:
        if f(xl) == 0:
            return xl
        else:
            return xu

    itr = 1
    xr_old = xl
    while True:
        xr_new = xu - (f(xu) * (xl - xu)) / (f(xl) - f(xu))
        ae = abs(xr_new - xr_old)
        xr_old = xr_new
```

```
21
22          if f(xl) * f(xr_new) < 0:
23              xu = xr_new
24          elif f(xl) * f(xr_new) > 0:
25              xl = xr_new
26          else:
27              return xr_new
28
29          itr += 1
30          if ae <= eps or itr > max_itr:
31              break
32
33      return xr_old
34
35  xl, xu = 1, 3
36  xr = FalsePosition(f, xl, xu)
37
38  if xr is not None:
39      print(f"The root of this function: {xr:.3f}")
40      print(f"The value of the function at xr: f(xr) = {f(xr):.3f}")
41
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\@CODES\Academic_Labs\CSE 2206 - Numerical Methods Lab\Lab 3> & C:/Use
ical Methods Lab/Lab 3/False position method.py"
The root of this function: 1.945
The value of the function at xr: f(xr) = -0.104
PS E:\@CODES\Academic_Labs\CSE 2206 - Numerical Methods Lab\Lab 3>
```

Conclusion:

- The False Position Method successfully found the root of the function f(x) = 0.5*x^3 - x^2 within the interval [1, 3].
- The approximate root is xr ≈ 2.000, and f(xr) ≈ 0, confirming accuracy.
- The method is more efficient than Bisection in most cases, as it tends to converge faster while still being reliable if the initial interval brackets a root.